



Современные архитектуры НС



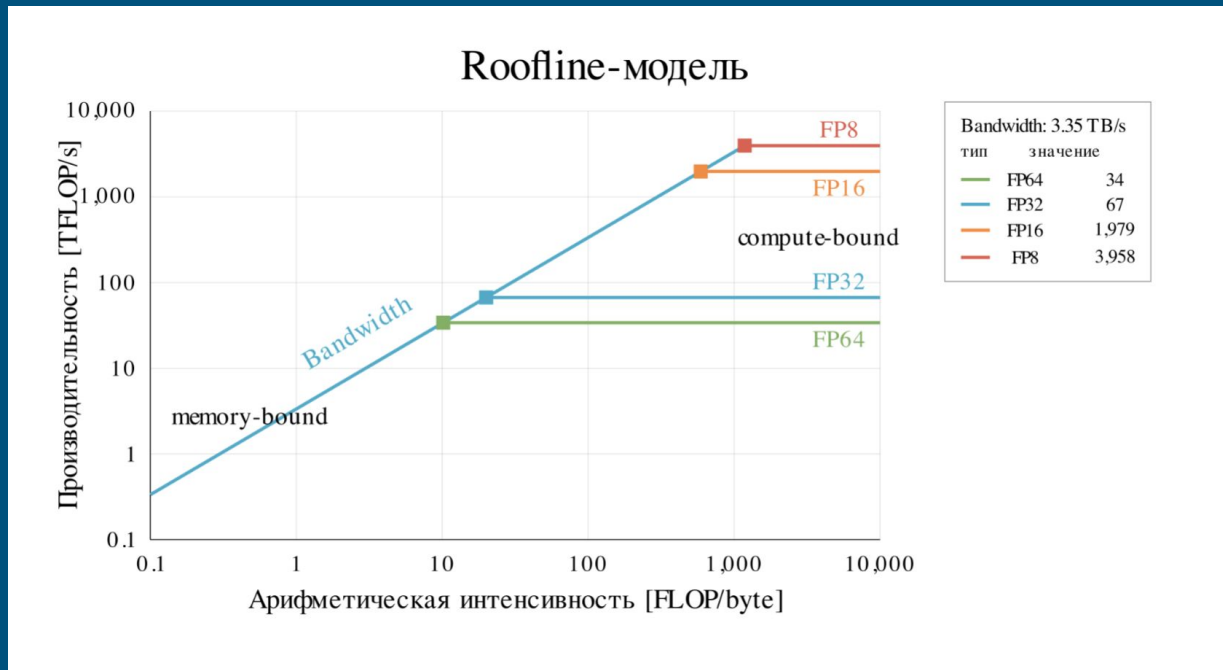
Лекция 4 Flash Attention



План лекции

1. **Оптимизация вычислений нейронных сетей на GPU**
2. Flash Attention
3. Triton
4. Профилирование

Roofline-модель



Тайлинг GEMM

Основная операция в нейронных сетях (90% всех FLOPs) — это GEMM, поэтому большую часть оптимизаций посвятим ей.

У GEMM есть потенциал стать compute-bound операцией, просто мы неправильно ее считаем.

$$C = AB,$$

где

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

Тайлинг GEMM

В наивном GEMM мы считываем из памяти строку матрицы A и столбец матрицы B, после чего вычисляем скалярное произведение

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}$$

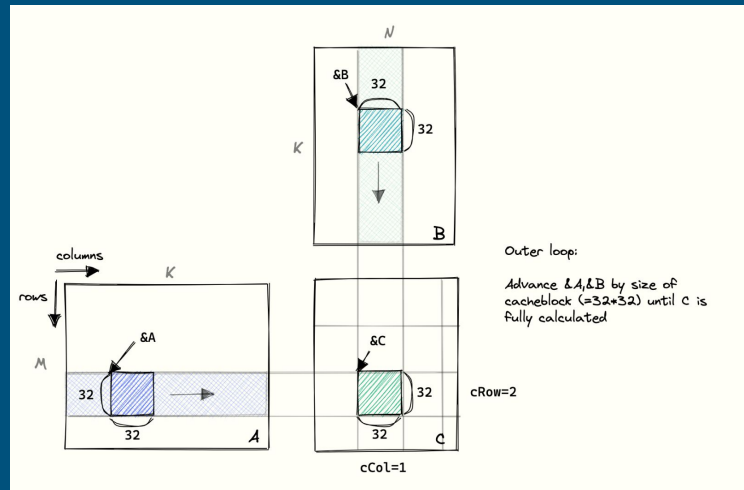
На две операции, сложение и умножение, необходимо прочесть 8 байт. Арифметическая интенсивность на дне.

Тайлинг GEMM

Снизим количество обращений к глобальной памяти за счёт того, что будем кэшировать некоторые элементы в разделяемой памяти.

Для этого разобьем вычисление на блоки и будем грузить их в SMEM.

Это называется **тайлинг**.



$$C_{I,J} = \sum_T A_{I,T} B_{T,J}$$

Арифметическая интенсивность тайлинга

Пока всё выглядит так, будто мы просто раздули количество обращений к памяти.

Арифметическая интенсивность тайлинга

Пока всё выглядит так, будто мы просто раздули количество обращений к памяти.

Не кажется.

Арифметическая интенсивность тайлинга

Пока всё выглядит так, будто мы просто раздули количество обращений к памяти.

Не кажется. Так и есть.

Арифметическая интенсивность тайлинга

Пока всё выглядит так, будто мы просто раздули количество обращений к памяти.

Не кажется. Так и есть. **Но только формально.**

Арифметическая интенсивность тайлинга

Пока всё выглядит так, будто мы просто раздули количество обращений к памяти.

Не кажется. Так и есть. Но только формально.

Вот арифметическая интенсивность наивного GEMM

$$I_{\text{naive}} = \frac{2 \text{ FLOP}}{2 \cdot 4 \text{ bytes}} = \frac{1}{4} \text{ FLOP/byte}$$

Арифметическая интенсивность тайлинга

Найдем арифметическую интенсивность тайлинга

Для вычисления одной составляющей тайла количество FLOPs равно:

$$\text{FLOP}_{\text{tile}} = 2B_M B_N B_K$$

А считать надо два тайла из GEMM и считать элементы из SMEM:

$$\text{Bytes}_{\text{GMEM}} = 4(B_M B_K + B_K B_N)$$

$$\text{Bytes}_{\text{SMEM}} = 2 \cdot 4B_M B_N B_K$$

Арифметическая интенсивность тайлинга

Арифметическая интенсивность тайлинга будет равна:

$$I_{\text{tilled}} = \frac{2B_M B_N B_K}{\text{Bytes}_{\text{GMEM}} + \text{Bytes}_{\text{SMEM}}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N) + 2 \cdot 4B_M B_N B_K}$$

Арифметическая интенсивность тайлинга

Арифметическая интенсивность тайлинга будет равна:

$$I_{\text{tiled}} = \frac{2B_M B_N B_K}{\text{Bytes}_{\text{GMEM}} + \text{Bytes}_{\text{SMEM}}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N) + 2 \cdot 4B_M B_N B_K}$$

Упростим:

$$I_{\text{tiled}} = \frac{1}{2/B_N + 2/B_M + 4} \leq \frac{1}{4} \text{ FLOP/byte}$$

Арифметическая интенсивность тайлинга

Арифметическая интенсивность тайлинга будет равна:

$$I_{\text{tilled}} = \frac{2B_M B_N B_K}{\text{Bytes}_{\text{GMEM}} + \text{Bytes}_{\text{SMEM}}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N) + 2 \cdot 4B_M B_N B_K}$$

Упростим:

$$I_{\text{tilled}} = \frac{1}{2/B_N + 2/B_M + 4} \leq \frac{1}{4} \text{ FLOP/byte}$$

Формально — всё плохо. На деле — всё отлично.

Арифметическая интенсивность тайлинга

На практике чтения из SMEM можно не учитывать

Тогда:

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N)} = \frac{1}{2/B_N + 2/B_M}$$

Видно, что чем больше тайлы — тем выше арифметическая интенсивность

Повышение арифметической интенсивности

К сожалению, размер блока ограничен 1024 потоками. Это значит, что мы можем читать только.

Повышение арифметической интенсивности

К сожалению, размер блока ограничен 1024 потоками. Это значит, что мы можем читать только.

При таком ограничении мы добъёмся максимальной арифметической интенсивности только при тайлинге 32x32.

Повышение арифметической интенсивности

К сожалению, размер блока ограничен 1024 потоками. Это значит, что мы можем читать только.

При таком ограничении мы добъёмся максимальной арифметической интенсивности только при тайлинге 32×32 .

Однако, кто нам запрещает одним потоком вычислять несколько элементов конечной матрицы?

Повышение арифметической интенсивности

К сожалению, размер блока ограничен 1024 потоками. Это значит, что мы можем читать только.

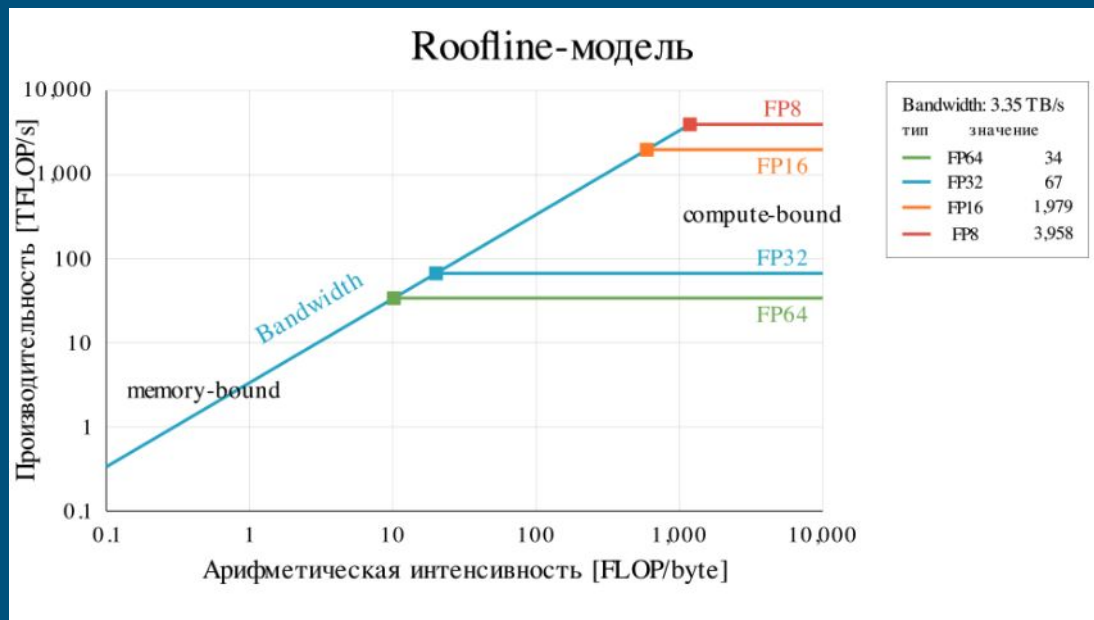
При таком ограничении мы добьёмся максимальной арифметической интенсивности только при тайлинге 32×32 .

Однако, кто нам запрещает одним потоком вычислять несколько элементов конечной матрицы?

Запрещает регистровое давление

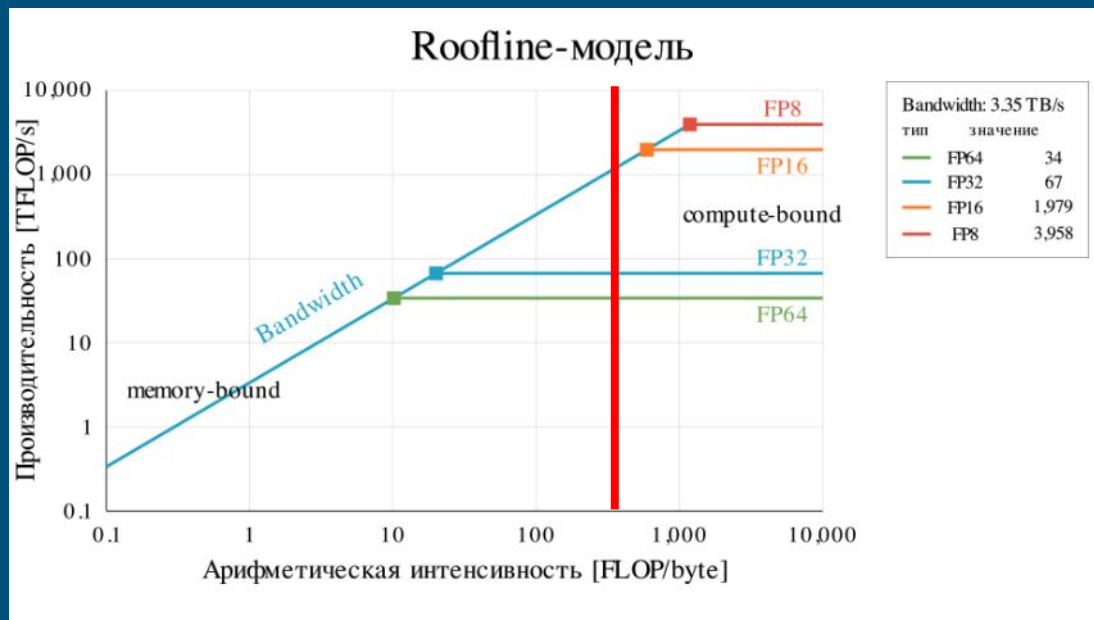
Вычисления в пониженной точности

Снова вспомним roofline-модель



Вычисления в пониженной точности

Снова вспомним roofline-модель



Вычисления в пониженной точности

Вспомним нашу арифметическую интенсивность тайлинга:

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2}{4(1/B_N + 1/B_M)}$$

4 в знаменателе — это количество байт, необходимые для записи одного числа в типе float32

Вычисления в пониженной точности

Число в “полной” точности представляется следующим образом:

- 1 бит — знак s
- 8 бит — показатель степени e
- 23 бита — мантисса m

Нормализованное значение вычисляется по формуле

$$(-1)^s \cdot 2^{e-\text{bias}} \cdot 1.m$$

Вычисления в пониженной точности

Разумным решением будет снизить точность и использовать для хранения float16:

- 1 бит — знак
- 5 бит — показатель степени
- 10 бит — мантисса

Вычисления в пониженной точности

Уменьшив точность float, мы одновременно и уходим от memory-bound, увеличивая арифметическую интенсивность (а также снижаем давление на RMEM и SMEM)

$$I_{\text{tiled, fp16}}^{\text{GMEM}} = \frac{2}{2(1/B_N + 1/B_M)} = \frac{1}{1/B_N + 1/B_M}$$

И при этом отодвигаем compute-bound предел.

Переполнение и округление float

Используя пониженную точность, нам также требуется меньше памяти для хранения параметров, градиентов, состояния оптимизатора и активация.

Поэтому мы также можем обучать более крупные модели.

Переполнение и округление float

Однако у float16 есть существенный недостаток — у него меньше бит под экспоненту, это существенно сужает диапазон представимых значений

`float32` : $\approx 1.18 \cdot 10^{-38} \dots 3.4 \cdot 10^{38}$

`float16` : $\approx 6.10 \cdot 10^{-5} \dots 6.55 \cdot 10^4$

Это может быть критично при выполнении операций с элементами с большой магнитудой и может привести к **переполнению типа**.

Переполнение и округление float

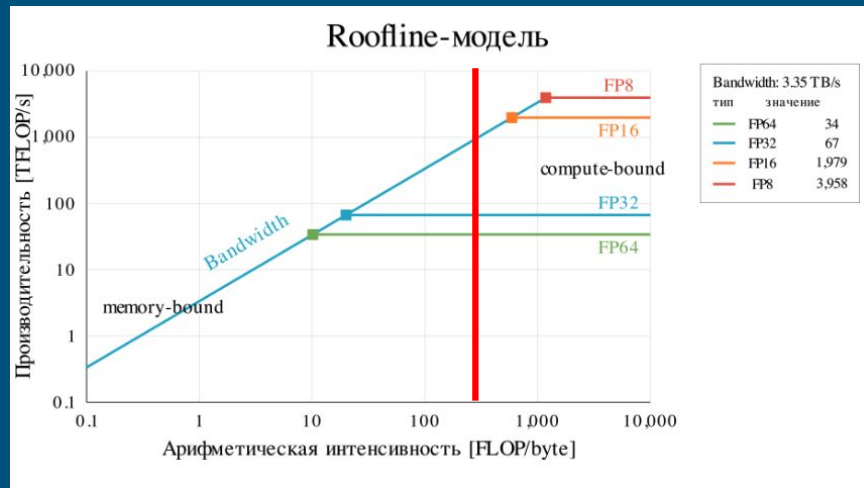
Чтобы избежать подобной ситуации, был разработан тип данных bfloat16:

- 1 бит — знак
- 8 бит — показатель степени
- 7 бит — мантисса

Тензорные ядра в GEMM

Мы достигли высокого результата арифметической интенсивности GEMM. Операция — compute-bound.

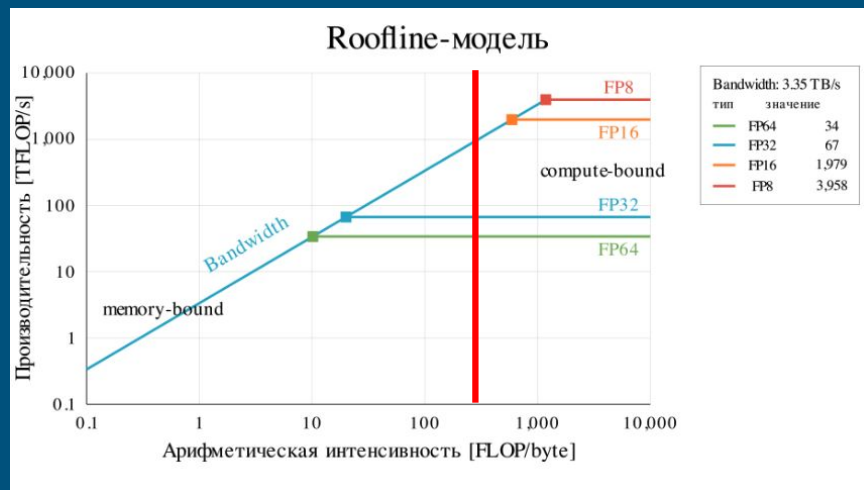
Один поток считает подтайл размером 4x4.



Тензорные ядра в GEMM

Мы достигли высокого результата арифметической интенсивности GEMM. Операция — compute-bound.

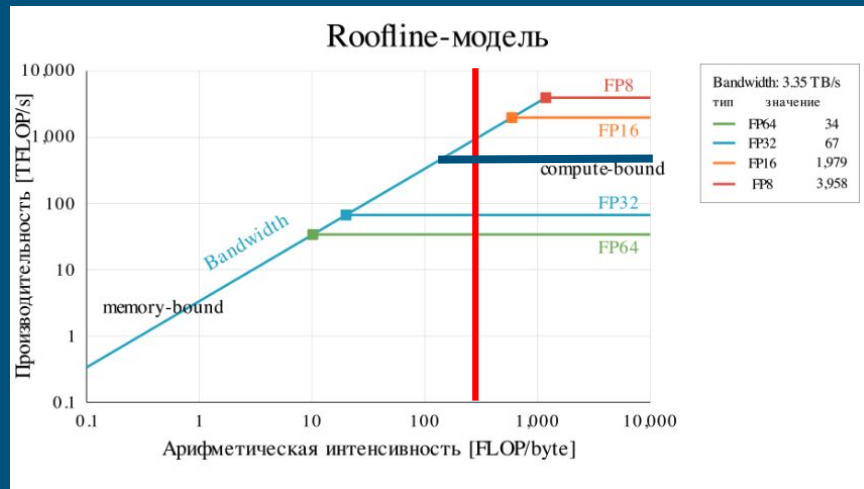
Один поток считает подтайл размером 4x4. Последовательно.



Тензорные ядра в GEMM

Мы достигли высокого результата арифметической интенсивности GEMM. Операция – compute-bound.

Один поток считает под-тайл размером 4x4. Последовательно.

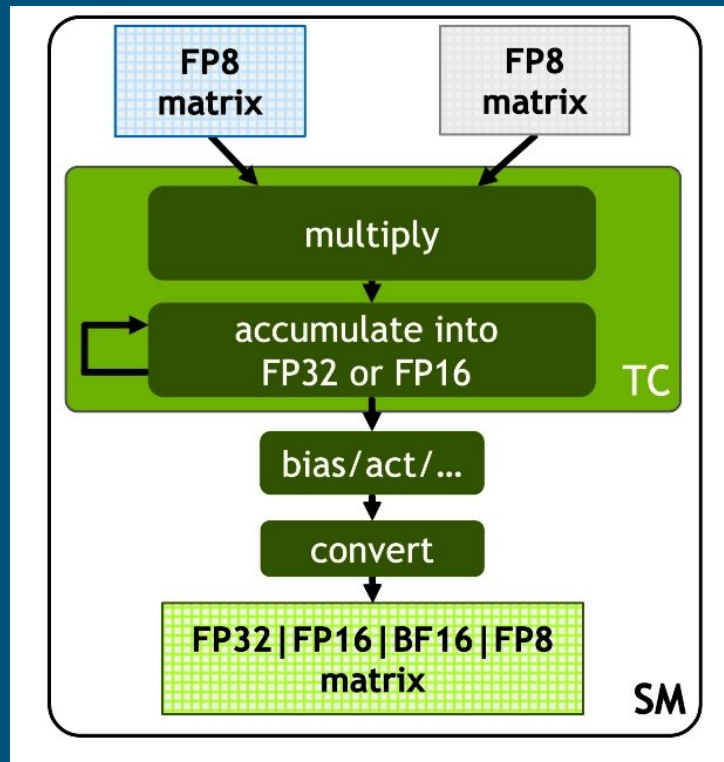


Тензорные ядра в GEMM

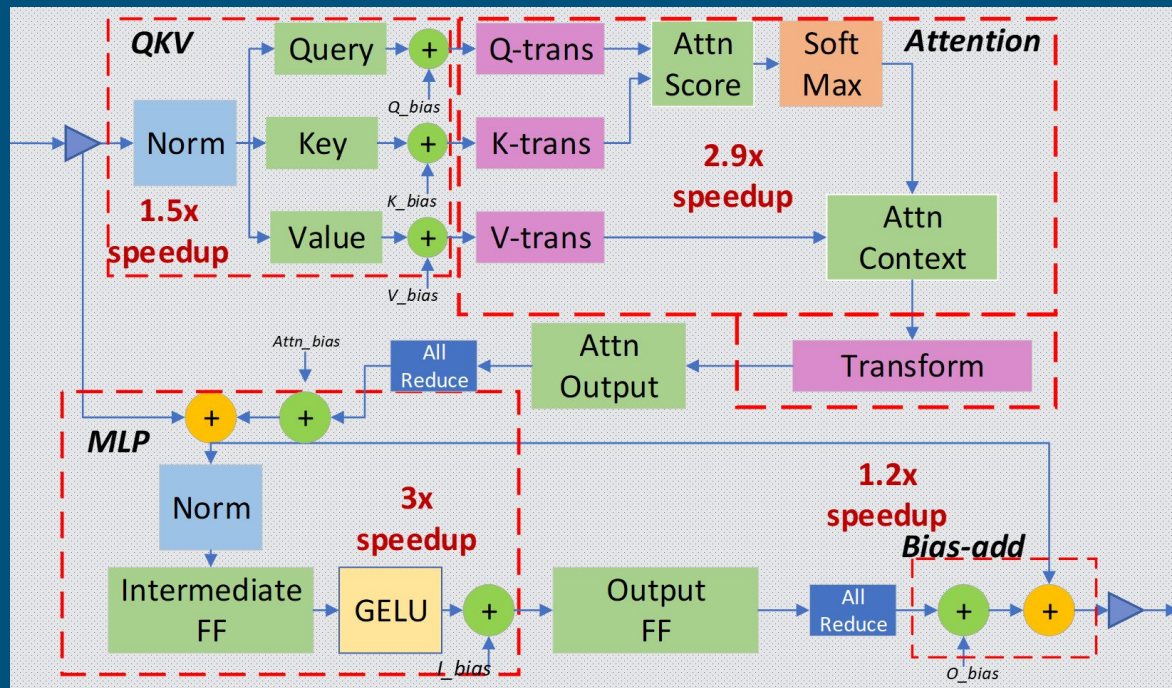
В GPU есть аппаратные блоки — тензорные ядра. Они предназначены для быстрого выполнения MMA-операции:

$$C_{I,J}^R \leftarrow C_{I,J}^R + A_{I,T}^R B_{T,J}^R$$

Работа с данными остается той же, но вычисления происходят эффективнее



Kernel Fusion



План лекции

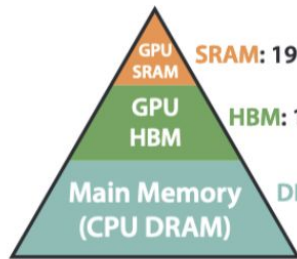
1. Оптимизация вычислений нейронных сетей на GPU
2. **Flash Attention**
3. Triton
4. Профилирование

Проблемы наивного внимания

Рассмотрим одну голову внимания. В наивной реализации вычисление механизма внимания будет разбито на несколько этапов:

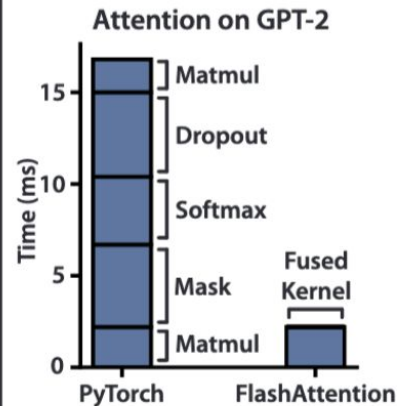
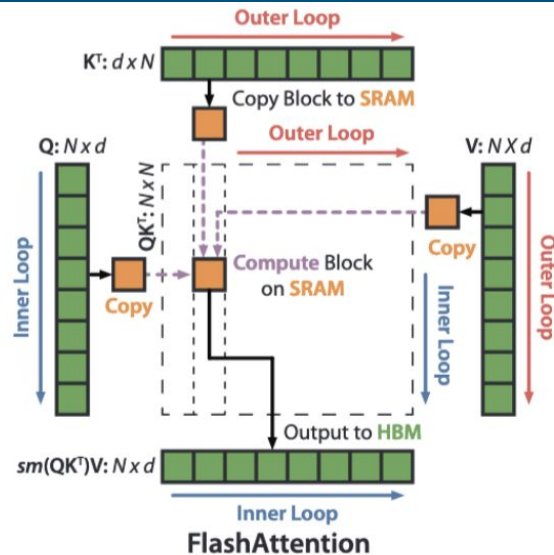
1. Вычислить $S = QK^T / \sqrt{d}$, после чего результат записать в GMEM;
2. Вычислить $A = \text{Softmax}(S)$, после чего результат также записать в GMEM;
3. Вычислить $O = AV$, после чего результат также записать в GMEM.

Идея Flash Attention



Memory Hierarchy with Bandwidth & Memory Size

SRAM: 19 TB/s (20 MB)
HBM: 1.5 TB/s (40 GB)
DRAM: 12.8 GB/s (>1 TB)



Safe Softmax

В обычном случае, Softmax применяется к строке следующим образом:

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Но мы выполняем вычисления в пониженной точности и не можем позволить разрастаться промежуточным значениям.

Safe Softmax

В обычном случае, Softmax применяется к строке следующим образом:

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Но мы выполняем вычисления в пониженной точности и не можем позволить разрастаться промежуточным значениям.

А в Softmax у нас сумма экспонент

Safe Softmax

Воспользуемся инвариантностью Softmax к сдвигу

$$\text{Softmax}(x - m)_i = \frac{\exp(x_i - m)}{\sum_{j=1}^N \exp(x_j - m)} = \frac{\exp(-m) \exp(x_i)}{\exp(-m) \sum_{j=1}^N \exp(x_j)} = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Чтобы избежать больших значений, будем вычитать из каждого значения максимум по строке

$$m = \max_j x_j$$

Online Softmax

Во Flash Attention мы хотим обрабатывать матрицу по блокам. Поэтому нужно научиться пересчитывать знаменатель и максимум в Softmax постепенно, по мере обработки блоков

$$m = \max_j x_j, \quad l = \sum_j \exp(x_j - m)$$

$$\text{Softmax}(x)_i = \frac{\exp(x_i - m)}{l}$$

Online Softmax

Пусть после $t-1$ блоков у нас уже есть максимум и частичная сумма знаменателя

$$l^{(t-1)} = \sum_{\text{old}} \exp(x_j - m^{(t-1)}).$$

Посчитаем максимум по строкам этого блока и обновим максимум по всем обработанным блокам

Online Softmax

Далее обновим знаменатель. Сначала посчитаем сумму по текущему блоку с новым максимумом

$$\hat{l}^{(t)} = \sum_{x_j \in x^{(t)}} \exp(x_j - m^{(t)})$$

Online Softmax

Далее обновим знаменатель. Сначала посчитаем сумму по текущему блоку с новым максимумом

$$\hat{l}^{(t)} = \sum_{x_j \in x^{(t)}} \exp(x_j - m^{(t)})$$

И обновим всю сумму, нормировав старое значение по новому максимуму

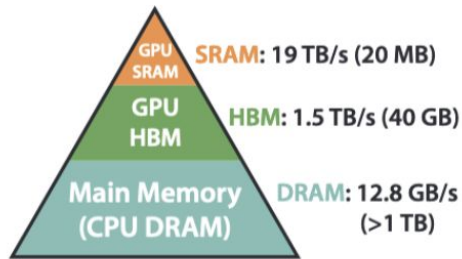
$$l^{(t)} = \frac{\exp(m^{(t-1)})}{\exp(m^{(t)})} l^{(t-1)} + \hat{l}^{(t)}$$

Online Softmax

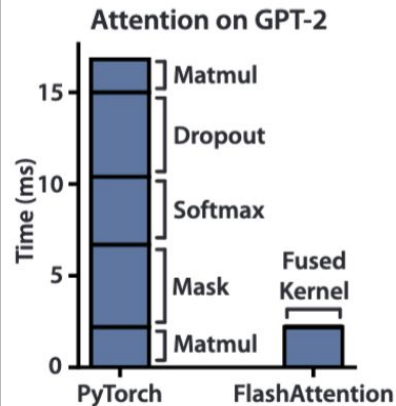
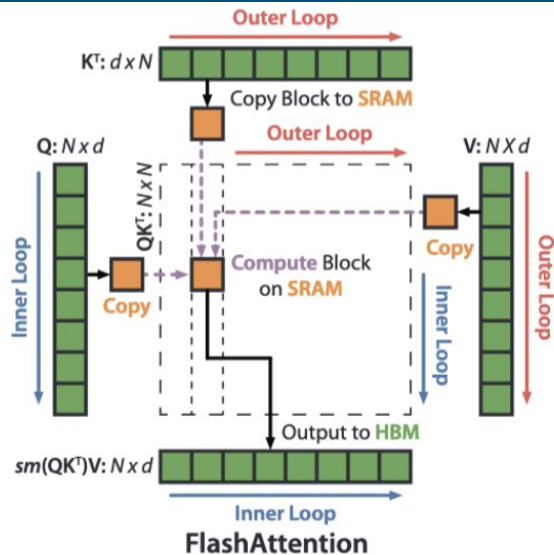
Если упростить:

$$l^{(t)} = \exp(m^{(t-1)} - m^{(t)})l^{(t-1)} + \hat{l}^{(t)}$$

QKV-тайлинг



Memory Hierarchy with Bandwidth & Memory Size



QKV-тайлинг

Матрицу весов внимания можно записать таким образом:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)}$$

Или же:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{l(t)}$$

QKV-тайлинг

Вычислять матрицу весов внимания не будем, так как знаменатель на данный момент не имеет конечный вид.

Будем поддерживать следующую ненормированную сумму

$$U^{(t-1)} = \sum_{\text{old } j} \exp(S_{ij} - m^{(t-1)}) V_j$$

Обновим ее:

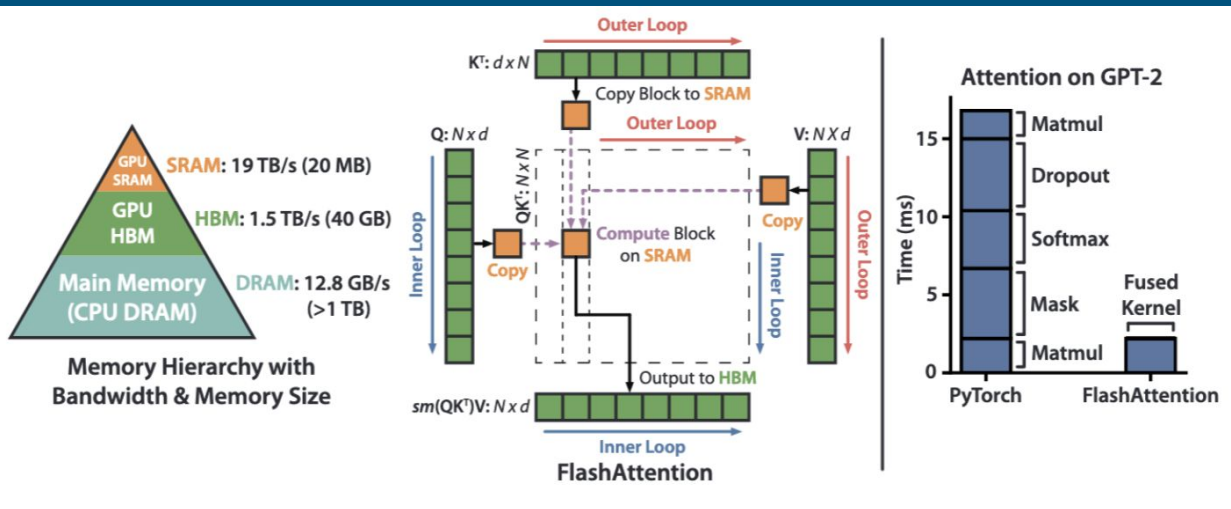
$$\hat{U}^{(t)} = \exp(S_{i,t} - m^{(t)}) V_t$$

$$U^{(t)} = \exp(m^{(t-1)} - m^{(t)}) U^{(t-1)} + \hat{U}^{(t)}$$

QKV-тайлинг

После обработки всех блоков
итоговый выход получается равным

$$O_i = \frac{U^{(T)}}{l^{(T)}}$$



Наивное внимание vs Flash Attention

Все эти шаги сохраняют лишние активации и съедают память квадратично

	seq_len	batch	heads	head_dim	dtype	block_m	block_n	torch_peak_extra_mb	triton_peak_extra_mb	memory_ratio
0	128	1	8	64	float16	16	64	1.500000	0.125	12.000000
1	128	1	8	64	float16	32	64	1.500000	0.125	12.000000
2	128	1	8	64	float16	64	64	1.500000	0.125	12.000000
3	256	1	8	64	float16	16	64	5.245117	0.250	20.980469
4	256	1	8	64	float16	32	64	5.245117	0.250	20.980469
5	256	1	8	64	float16	64	64	5.245117	0.250	20.980469
6	512	1	8	64	float16	16	64	18.000000	0.500	36.000000
7	512	1	8	64	float16	32	64	18.000000	0.500	36.000000
8	512	1	8	64	float16	64	64	18.000000	0.500	36.000000
9	1024	1	8	64	float16	16	64	68.245117	1.000	68.245117
10	1024	1	8	64	float16	32	64	68.245117	1.000	68.245117
11	1024	1	8	64	float16	64	64	68.245117	1.000	68.245117
12	2048	1	8	64	float16	16	64	264.000000	2.000	132.000000
13	2048	1	8	64	float16	32	64	264.000000	2.000	132.000000
14	2048	1	8	64	float16	64	64	264.000000	2.000	132.000000

План лекции

1. Оптимизация вычислений нейронных сетей на GPU
2. Flash Attention
3. **Triton**
4. Профилирование

Зачем нужен Triton?

Писать высокопроизводительные
кernels на CUDA — сложно.

В PyTorch уже есть некоторые
основные операции и модули,
оптимизированные наилучшим
образом.

Однако, если мы решим сделать
какой то свой механизм, то он вряд
ли будет реализован в PyTorch.

Зачем нужен Triton?

Писать высокопроизводительные ядра на CUDA — сложно.

В PyTorch уже есть некоторые основные операции и модули, оптимизированные наилучшим образом.

Однако, если мы решим сделать какой-то свой механизм, то он вряд ли будет реализован в PyTorch.



Кернел в Triton и Program Instance

```
@triton.jit
def vector_add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask, other=0)
    y = tl.load(y_ptr + offsets, mask=mask, other=0)

    tl.store(out_ptr + offsets, x + y, mask=mask)
```

Кернел в Triton и Program Instance

```
pid = 0 :   offset = (0  1  2  3) ,  
pid = 1 :   offset = (4  5  6  7) ,  
pid = 2 :   offset = (8  9 10 11) ,  
  
...
```

SIMD

Один program instance вычисляет сразу блок итогового вектора (а не один элемент как обычно было в CUDA).

По сравнению с потоком в CUDA — экземпляр программы тритона это более крупная логическая единица.

Иными словами, тритон использует блочную векторизованную модель программирования. По форме она похоже на **SIMD**, но на практике выполняется она на SIMT-подобной архитектуре.

План лекции

1. Оптимизация вычислений нейронных сетей на GPU
2. Flash Attention
3. Triton
4. **Профилирование**

Бенчмарк

	seq_len	batch	heads	head_dim	dtype	block_m	block_n	torch_ms	triton_ms	speedup
0	128	1	8	64	float16	16	64	0.117292	0.099842	1.174770
1	128	1	8	64	float16	32	64	0.117292	0.040311	2.909693
2	128	1	8	64	float16	64	64	0.117292	0.053360	2.198124
3	256	1	8	64	float16	16	64	0.094909	0.150760	0.629540
4	256	1	8	64	float16	32	64	0.094909	0.129091	0.735212
5	256	1	8	64	float16	64	64	0.094909	0.096170	0.986886
6	512	1	8	64	float16	16	64	0.356698	0.545544	0.653839
7	512	1	8	64	float16	32	64	0.356698	0.482941	0.738596
8	512	1	8	64	float16	64	64	0.356698	0.374082	0.953530
9	1024	1	8	64	float16	16	64	1.259535	2.113808	0.595861
10	1024	1	8	64	float16	32	64	1.259535	1.790554	0.703433
11	1024	1	8	64	float16	64	64	1.259535	1.418573	0.887888
12	2048	1	8	64	float16	16	64	5.545779	8.178800	0.678068
13	2048	1	8	64	float16	32	64	5.545779	6.846865	0.809973
14	2048	1	8	64	float16	64	64	5.545779	5.099588	1.087495

Закон Амдала

Важно различать бенчмарк одной отдельной операции и полный замер всего шага обучения.

Ускорение в первом случае может быть почти не заметно на всём масштабе. Это описывает закон Амдала:

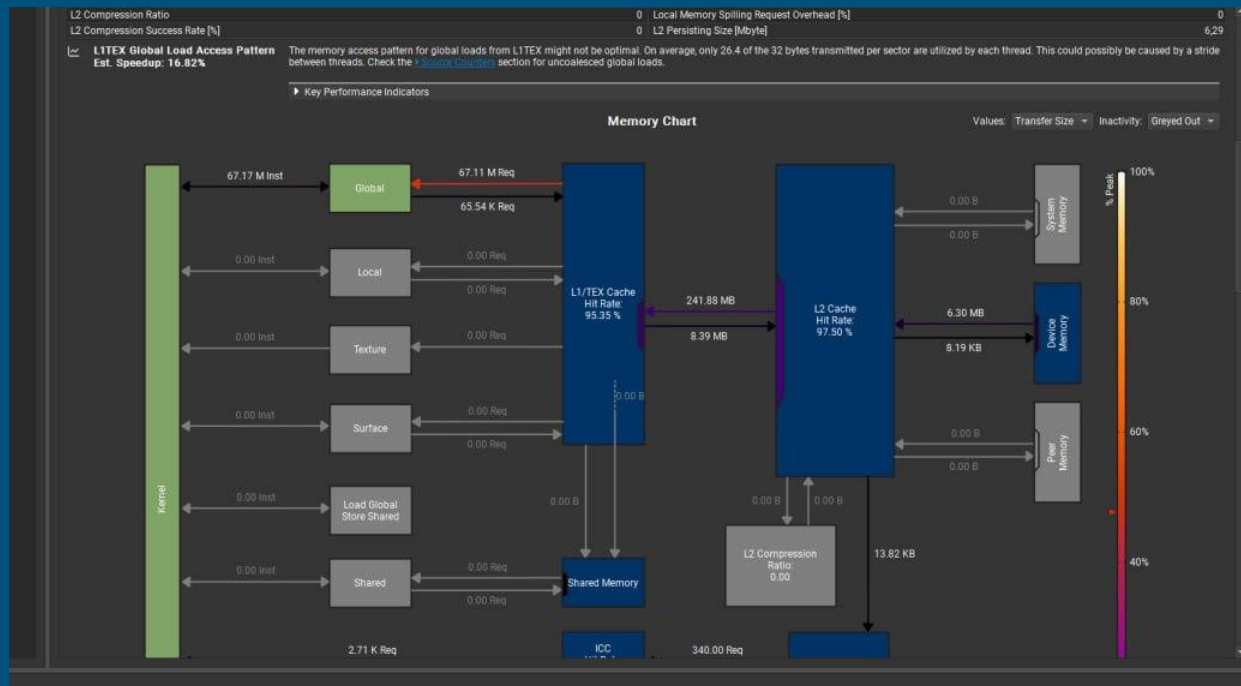
$$S = \frac{1}{(1 - p) + \frac{p}{s}}$$

Закон Амдала

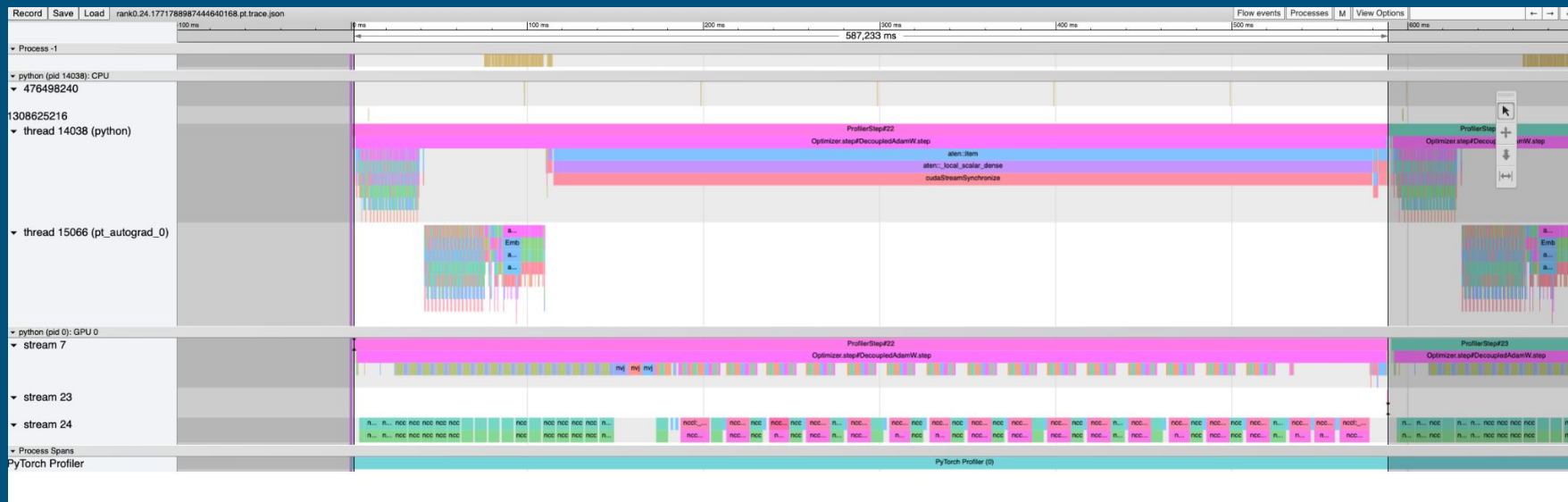
Пусть например, операция занимала 10% времени шага обучения. А мы ускорили ее в 10 раз. Тогда общее ускорение равно:

$$S = \frac{1}{0.9 + 0.01} \approx 1.1$$

Профилирование обучения



Профилирование кернелов



Вопросы?
